

Object Oriented Programming

Java



Object Oriented Programming

Two Paradigms

Process-oriented Model:

- Characterizes a program as a series of linear steps (that is, code).
- Code acting on data.
- Problems appear as programs grow larger and more complex.

Two Paradigms

Object-oriented Programming:

- Organizes a program around its data (that is, objects) and a set of well-defined interfaces to that data.
- Data controlling access to code.
- Manages increasing complexity.

OOP

- Object Oriented programming (OOP) is a programming paradigm.
- It is based on the concept of classes and objects.
- It structures a software program into simple, reusable pieces.
- Example: Java, Python, C#, C++ etc.

Advantages of OOP

- OOP simplifies structures
- Reusable, OOP objects can be used across programs
- Easier to debug, classes often contain all applicable information to them
- Secure, protects information through encapsulation

Building Blocks of OOP

- Classes
- Objects
- Methods
- Attributes

Class

- Classes are user defined data types.
- Collection of objects is called class.
- Class are a blueprint or a set of instructions to build a specific type of object.

Object

- Objects are instances of classes.

Method

- Methods represent behaviors.
- Methods perform actions; methods might return information about an object, or update an object's data.

Attribute

- Attributes are the information that is stored.
- Attributes are defined in the Class template.

A Code Snippet in Java

```
class Fruit {  
    life = 0;  
  
    constructor(name) {  
        this.name = name;  
    }  
  
    getTaste() {  
        // Check taste  
        return taste;  
    }  
  
    updateLife() {  
        this.life++;  
    }  
}
```

Abstraction

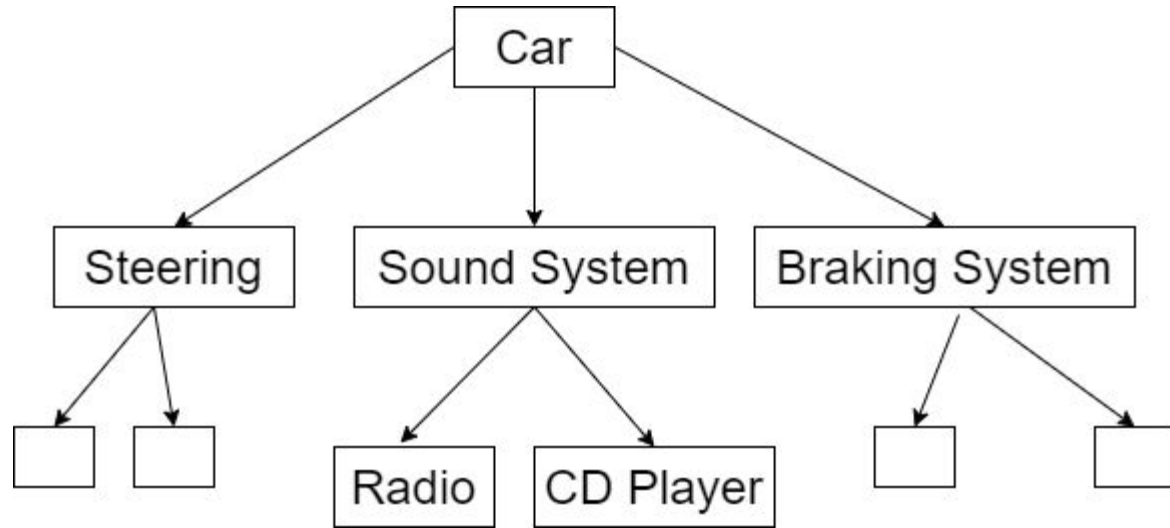
- An essential element of object-oriented programming.
- A powerful way to manage abstraction is through the use of hierarchical classifications.
- This allows to layer the semantics of complex systems, breaking them into more manageable pieces.

Abstraction

- The data from a traditional process-oriented program can be transformed by abstraction into its component objects.
- A sequence of process steps can become a collection of messages between these objects.
- Each of these objects describes its own unique behavior.

Abstraction

- We can treat these objects as concrete entities that respond to messages telling them to do something.



Principles of OOP

- Inheritance
- Encapsulation
- Polymorphism

Encapsulation

- Encapsulation is the mechanism that binds together code and the data it manipulates.
- keeps both safe from outside interference and misuse.
- A protective wrapper that prevents the code and data from being arbitrarily accessed by other code defined outside the wrapper.

Inheritance

- Inheritance is the process by which one object acquires the properties of another object.
- It supports the concept of hierarchical classification.

Polymorphism

- Polymorphism is a feature that allows one interface to be used for a general class of actions.
- The specific action is determined by the exact nature of the situation.

IntroToJava

Terms

- **Compiler**, computer software that translates (compiles) source code written in a high-level language (e.g., C++) into a set of machine-language instructions that can be understood by a digital computer's CPU.
- Code is **robust** if it tends to behave in a useful, controlled way, even when things go wrong.

Terms

- **Member** variables are known as instance variables in java.
Instance variables are declared in a class, but outside a method, constructor or any block.

Java's Magic: The Bytecode

- The output of a Java compiler is not executable code. Rather, it is bytecode.
- Bytecode is a highly optimized set of instructions designed to be executed by the Java run-time system, which is called the Java Virtual Machine (JVM).

Java's Magic: The Bytecode

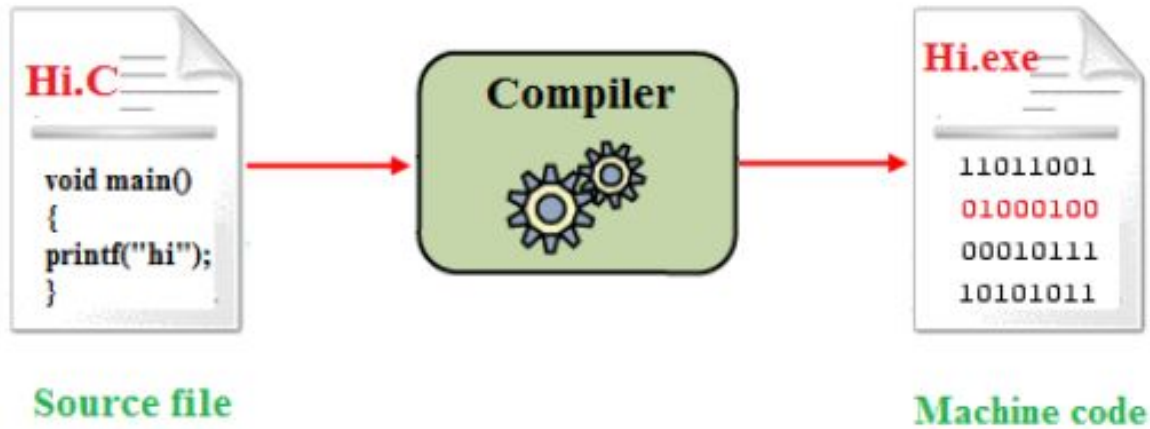
- Translating a Java program into bytecode makes it much easier to run a program in a wide variety of environments because only the JVM needs to be implemented for each platform.
- If the run-time package exists for a given system, any Java program can run on it.

Java's Magic: The Bytecode

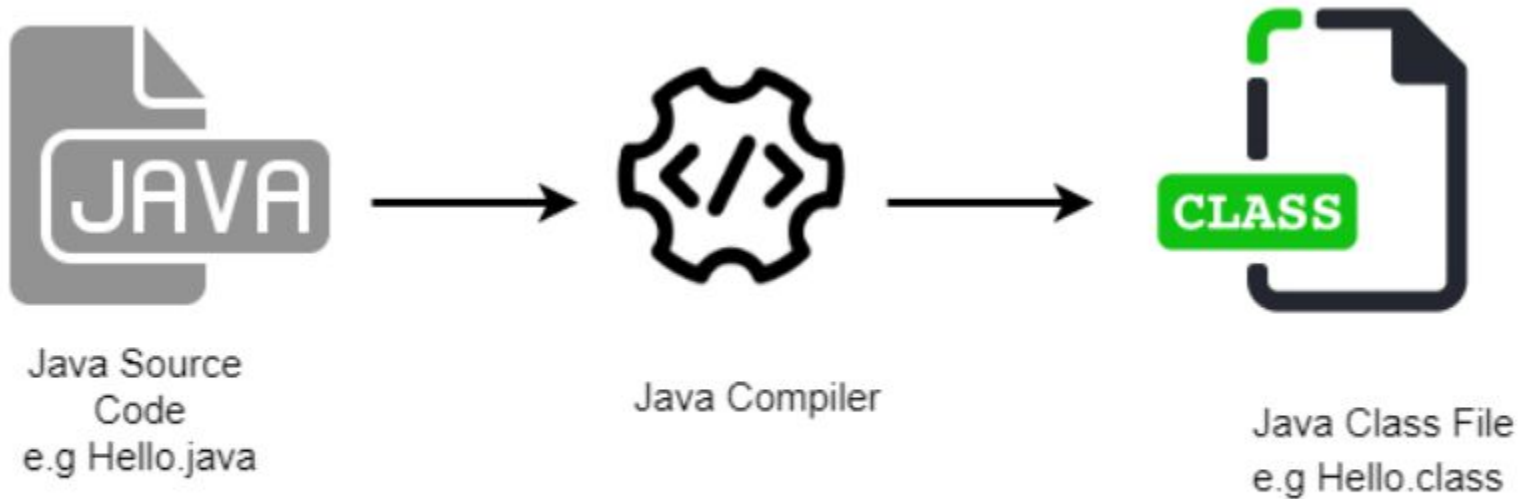
- If a Java program were compiled to native code, then different versions of the same program would have to exist for each type of CPU connected to the Internet. Thus, the execution of bytecode by the JVM is the easiest way to create truly portable programs.

Java Virtual Machine

- A specification that provides runtime environment in which java bytecode can be executed.
- Platform dependent means different OS needs different JVM.



Compiler



.class file contains **BYTECODE**

JAVA Compiler

The Java Buzzwords

The fundamental forces that necessitated the invention of Java are **portability** and **security**. Other Key considerations are as follows:

- Simple
- Secure
- Portable
- Object-oriented
- Robust
- Multithreaded
- Architecture-neutral
- Interpreted
- High performance
- Distributed
- Dynamic

The Java Buzzwords

Simple: Easy for the professional programmer to learn and use effectively.

Secure: Java is a secure language.

Portable: JVM makes Java portable.

Object-oriented: Java is an object oriented programming language.

Robust: Garbage collection, object oriented exception handling.

The Java Buzzwords

Multithreaded: Java supports multithreaded programming, which allows to write programs that do many things simultaneously.

Architecture-Neutral: “write once; run anywhere, any time, forever.”

Interpreted and High Performance: Java bytecode was carefully designed so that it would be easy to translate directly into native machine code for very high performance by using a just-in-time compiler.

The Java Buzzwords

Distributed: Java is designed for the distributed environment of the Internet.

Dynamic: Java is considered to be more dynamic since it is designed to adapt to an evolving environment.

A Simple Java Program

```
/*  
    This is a simple Java program.  
    Call this file "Example.java".  
*/  
class Example {  
    // Your program begins with a call to main().  
    public static void main(String args[]) {  
        System.out.println("This is a simple Java program.");  
    }  
}
```

Creating Java Program

- Any text editor or Java IDE (Integrated Development Environment) can be used to develop Java programs.
- Most widely used Java IDEs are:
 - Eclipse
 - IntelliJ IDEA
 - NetBeans

Compiling & Executing Java Program

javac Example.java

- Compiles the source file(transforms the source code into bytecodes)
- The bytecode is contained in the file named Example.class

java Example

- The JVM will look for the .class file and main method inside the class.If founded, it will execute the body of the main method otherwise it will generate an error.

The if Statement

- `if(condition) statement;`
- `if(num < 100) System.out.println("num is less than 100");`

Operator	Meaning
<	Less than
>	Greater than
==	Equal to

The For Loop

- `for(initialization; condition; iteration)` statement;
- the initialization portion of the loop sets a loop control variable to an initial value. The condition is a Boolean expression that tests the loop control variable.
- The iteration expression determines how the loop control variable is changed each time the loop iterates.

The For Loop

```
/*  
    Demonstrate the for loop.  
  
    Call this file "ForTest.java".  
*/  
class ForTest {  
    public static void main(String args[]) {  
        int x;  
  
        for(x = 0; x<10; x = x+1)  
            System.out.println("This is x: " + x);  
    }  
}
```

Whitespace

- Java is a free-form language. This means that there is no need to follow any special indentation rules.
- There has to be at least one whitespace character between each token that was not already delineated by an operator or separator.
- In Java, whitespace is a space, tab, or newline.

Identifiers

- Identifiers are used to name things, such as classes, variables, and methods.
- An identifier may be any descriptive sequence of uppercase and lowercase letters, numbers, or the underscore and dollar-sign characters. (The dollar-sign character is not intended for general use.)
- They must not begin with a number.
- Java is case-sensitive, so `VALUE` is a different identifier than `Value`.

Examples of valid identifiers

AvgTemp	count	a4	\$test	this_is_ok
---------	-------	----	--------	------------

Invalid identifier names include these:

2count	high-temp	Not/ok
--------	-----------	--------

NOTE Beginning with JDK 8, the use of an underscore by itself as an identifier is not recommended.

Literals

Literal in Java is a representation of boolean, numeric, character, or string data.

A constant value in Java is created by using a literal representation of it. For example, here are some literals:

100	98.6	'X'	"This is a test"
-----	------	-----	------------------

Separators

Symbol	Name	Purpose
()	Parentheses	Used to contain lists of parameters in method definition and invocation. Also used for defining precedence in expressions, containing expressions in control statements, and surrounding cast types.
{ }	Braces	Used to contain the values of automatically initialized arrays. Also used to define a block of code, for classes, methods, and local scopes.
[]	Brackets	Used to declare array types. Also used when dereferencing array values.
;	Semicolon	Terminates statements.
,	Comma	Separates consecutive identifiers in a variable declaration. Also used to chain statements together inside a for statement.
.	Period	Used to separate package names from subpackages and classes. Also used to separate a variable or method from a reference variable.
::	Colons	Used to create a method or constructor reference. (Added by JDK 8.)

The Java Keywords

abstract	continue	for	new	switch
assert	default	goto	package	synchronized
boolean	do	if	private	this
break	double	implements	protected	throw
byte	else	import	public	throws
case	enum	instanceof	return	transient
catch	extends	int	short	try
char	final	interface	static	void
class	finally	long	strictfp	volatile
const	float	native	super	while

The Primitive Types

Java defines eight primitive/simple types of data: **byte**, **short**, **int**, **long**, **char**, **float**, **double**, and **boolean**.

Java defines 4 integer types:

- byte
- short
- int
- long

Integer

Name	Width	Range
long	64	−9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
int	32	−2,147,483,648 to 2,147,483,647
short	16	−32,768 to 32,767
byte	8	−128 to 127

Floating - Point

- float
- double

Name	Width In Bits	Approximate Range
double	64	4.9e-324 to 1.8e+308
float	32	1.4e-045 to 3.4e+038

Characters & Boolean

Characters:

- Keyword: char
- Needs 16 bit of storage.
- Represents character in Unicode.
- Can represents 65,536 symbols and character.

Boolean:

- Only have two possible values: true or false.

Non- primitive Data Types

The non-primitive data types in java are: Objects, Array

Arrays

- An array is a group of like-typed variables that are referred to by a common name.
- Arrays of any type can be created and may have one or more dimensions.
- A specific element in an array is accessed by its index.

Declaring & Initialization of Array in Java

Declaration:

- `dataType[] arr;`
- `dataType []arr;`
- `dataType arr[];`

Initialization:

- `int a[]=new int[5];`
- `int a[ ] = {1, 2, 3, 4, 5}`

```
public class ArrayDemo {  
    public static void main(String[] args) {  
        //      Array Declaration  
        //      int a[];  
        //      int[] a;  
        //      int []a;  
  
        //      Array initialization  
        //      int a[] = new int[]{1, 2, 3, 4, 5};  
        //      int a[] = {1,2,3,4,5};  
  
        //      int a[10]; //error  
        //      int a[] = new int{1, 2, 3, 4, 5}; // error  
  
        int[] a = new int[5];  
        for (int i = 0; i < a.length; i++) {  
            a[i] = i;  
        }  
  
        for (int i = 0; i < a.length; i++) {  
            System.out.println(a[i]);  
        }  
    }  
}
```

Arithmetic Operators

Operator	Result
+	Addition (also unary plus)
-	Subtraction (also unary minus)
*	Multiplication
/	Division
%	Modulus
++	Increment
+=	Addition assignment
- =	Subtraction assignment
*=	Multiplication assignment
/=	Division assignment
%=	Modulus assignment
--	Decrement

Bitwise Operator

Operator	Result
~	Bitwise unary NOT
&	Bitwise AND
	Bitwise OR
^	Bitwise exclusive OR
>>	Shift right
>>>	Shift right zero fill
<<	Shift left
&=	Bitwise AND assignment
=	Bitwise OR assignment
^=	Bitwise exclusive OR assignment
>>=	Shift right assignment
>>>=	Shift right zero fill assignment
<<=	Shift left assignment

Relational Operators

Operator	Result
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Boolean Logical Operators

Operator	Result
&	Logical AND
	Logical OR
^	Logical XOR (exclusive OR)
	Short-circuit OR
&&	Short-circuit AND
!	Logical unary NOT
&=	AND assignment
=	OR assignment
^=	XOR assignment
==	Equal to
!=	Not equal to
?:	Ternary if-then-else

Increment & Decrement Operator

```
public class IncrementDecrementOperator {  
    public static void main(String[] args) {  
        int x=5,y;  
        y = x++;  
        System.out.println(y);  
  
        y = ++x;  
        System.out.println(y);  
  
        y = --y;  
        System.out.println(y);  
    }  
}
```

Control Statements

- If statement
- If - else if chain
- If - else - if chain

Nested If Statement

```
if (expression1){  
    if (expression2){  
        // body of the child conditional  
    }  
}
```

For Loop

```
for(initialization; condition; iteration) {  
    // body  
}
```

For-Each Version of the for Loop

```
public class ForEachDemo {  
    public static void main(String[] args) {  
        int a[] = {10, 20, 30, 40, 50};  
        int query = 20;  
  
        for (int i : a) {  
            i = i * 2; // Changes i, not the array element  
            if (i == query) {  
                // Not aware about the index of the query  
            }  
            System.out.print(i + " ");  
        }  
        System.out.println();  
    }  
}
```

Limitations of For-Each

- Not appropriate for modifying the array.
- Do not keep track of index.
- Only iterates forward over the array in single steps

While Loop

```
while(condition) {  
    // body of loop  
    iteration;  
}
```

do-while Loop

```
do {  
    // body of loop  
    iteration;  
} while (condition);
```

Nested Loop

```
for(initialization1; condition1; iteration1) {  
    // body: Outer Loop  
    for(initialization2; condition2; iteration2) {  
        // body : Inner Loop  
    }  
}
```

Nested Loop (Contd.)

```
for(initialization1; condition1; iteration1) {  
    // body1  
    while(condition2) {  
        // body2  
    }  
}
```

Jump Statements

- Break:
 - It terminates a statement sequence in a switch statement.
 - It can be used to exit a loop.
 - It can be used as a “civilized” form of goto.

Jump Statement (Contd.)

- Continue:
 - In while and do-while loops, a continue statement causes control to be transferred directly to the conditional expression that controls the loop.
 - In a for loop, control goes first to the iteration portion of the for statement and then to the conditional expression.

Scanner

- The Scanner class is used to get user input, and it is found in the `java.util` package.
- Used for obtaining the input of the primitive types like `int`, `double`, etc. and strings.

Scanner

```
import java.util.Scanner;

public class ScannerDemo {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter a number: ");
        int num = scanner.nextInt();
        System.out.println(num);

        while (scanner.hasNextLine()) {
            System.out.println(scanner.nextLine());
        }
    }
}
```

Strings

- `String str = "Hello";`
- `String str = new String("Hello");`
- String objects are immutable.
- Immutable means unmodifiable or unchangeable.
- Once String object is created its data or state can't be changed but a new String object is created.

Strings (Contd.)

1. str → "Hello"
2. str → "Hello"
→ "Hi"

```
public class StringDemo {  
    public static void main(String[] args) {  
        String str = new String( original: "Hello");  
        str = "Hi";  
        System.out.println(str);  
  
        String str1 = "one";  
        String str2 = str1;  
        str1 = "two";  
        System.out.println(str2);  
    }  
}
```

Strings(Contd.)

The String class has a set of built-in methods.

https://www.w3schools.com/java/java_ref_string.asp

Classes & Objects

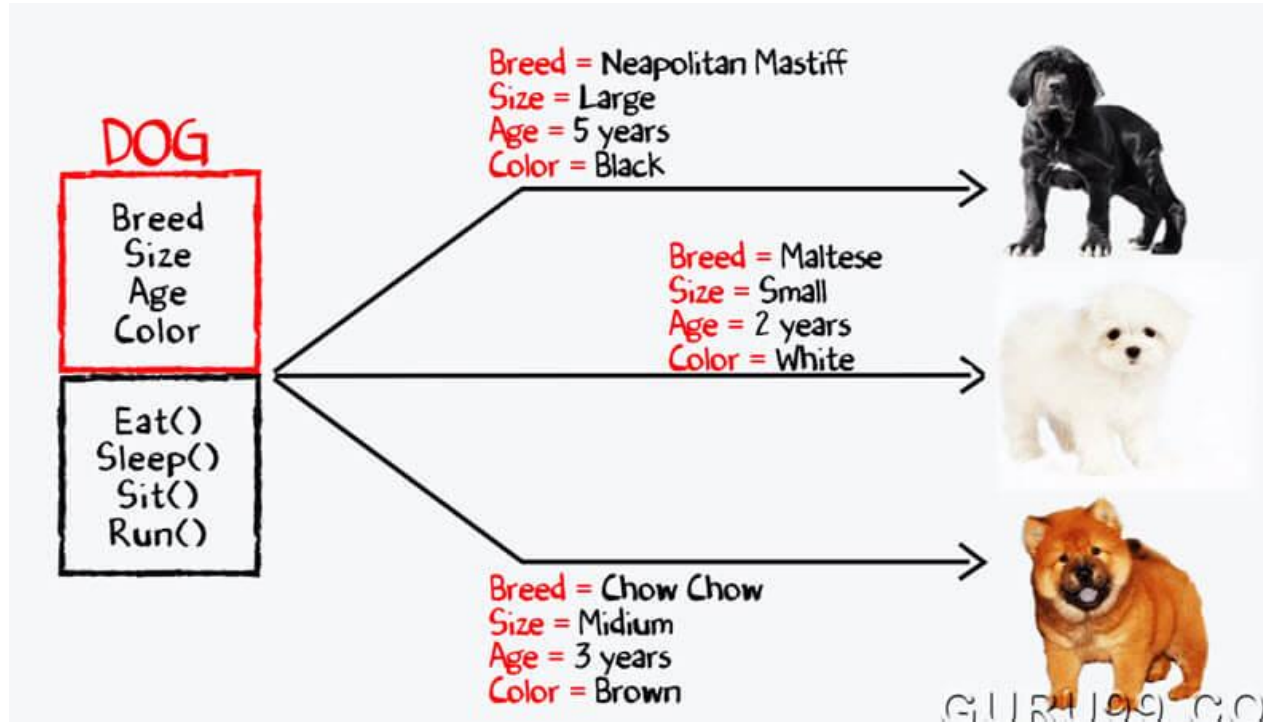
Introduction

- Classes define the shape and nature of an object.
- The class forms the basis for object-oriented programming in Java.
- Class defines a new data type. Once defined, this new type can be used to create objects of that type.
- A class is a template for an object, and an object is an instance of a class.

The General Form of a Class

- A class is declared by use of the **class** keyword.
- The data, or variables, defined within a class are called instance variables.
- The code is contained within methods.
- Collectively, the methods and variables defined within a class are called members of the class.
- It is the methods that determine how a class data can be used.
- Each instance of the class (that is, each object of the class) contains its own copy of these variables.

Class & Object



Class

```
class Box {  
    double width;  
    double height;  
    double depth;  
}
```

Class (Cont.)

- A class defines a new type of data. In this case, the new data type is called Box.
- We will use this name to declare objects of type Box.
- A class declaration only creates a template; it does not create an actual object.

Class (Cont.)

```
Box mybox = new Box(); // create a Box object called mybox  
mybox.width = 100;
```

Class (Cont.)

```
// This class declares an object of type Box.
class BoxDemo {
    public static void main(String args[]) {
        Box mybox = new Box();
        double vol;

        // assign values to mybox's instance variables
        mybox.width = 10;
        mybox.height = 20;
        mybox.depth = 15;

        // compute volume of box
        vol = mybox.width * mybox.height * mybox.depth;

        System.out.println("Volume is " + vol);
    }
}
```

```
class BoxDemo2 {
    public static void main(String args[]) {
        Box mybox1 = new Box();
        Box mybox2 = new Box();
        double vol;

        // assign values to mybox1's instance variables
        mybox1.width = 10;
        mybox1.height = 20;
        mybox1.depth = 15;

        /* assign different values to mybox2's
           instance variables */
        mybox2.width = 3;
        mybox2.height = 6;
        mybox2.depth = 9;

        // compute volume of first box
        vol = mybox1.width * mybox1.height * mybox1.depth;
        System.out.println("Volume is " + vol);

        // compute volume of second box
        vol = mybox2.width * mybox2.height * mybox2.depth;
        System.out.println("Volume is " + vol);
    }
}
```

Declaring Objects

```
Box mybox = new Box();
```

1. `Box mybox;` // declare reference to object
 2. `mybox = new Box();` // allocate a Box object
- Line 1 declares mybox as a reference to an object of type Box.
At this point, mybox does not yet refer to an actual object.
 - The next line allocates an object and assigns a reference to it to mybox.

new Operator

- The **new** operator dynamically allocates (that is, allocates at run time) memory for an object and returns a reference to it. This reference is, more or less, the address in memory of the object allocated by new. This reference is then stored in the variable.
- **new** allocates memory for an object during run time.

Declaring an object of type Box

Statement

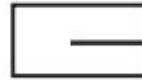
Effect

Box mybox;

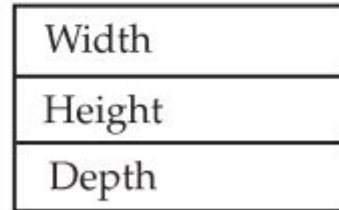


mybox

mybox = new Box();



mybox



Box object

A Closer Look at new

`class-var = new classname ( );`

- Here, class-var is a variable of the class type being created.
- The classname is the name of the class that is being instantiated.
- The class name followed by parentheses specifies the constructor for the class.
- A constructor defines what occurs when an object of a class is created.
- However, if no explicit constructor is specified, then Java will automatically supply a default constructor.

Class vs. Object

- A class creates a new data type that can be used to create objects. That is, a class creates a logical framework that defines the relationship between its members.
- Declaring an object of a class means creating an instance of that class.
- Thus, a class is a logical construct. An object has physical reality. (That is, an object occupies space in memory.)

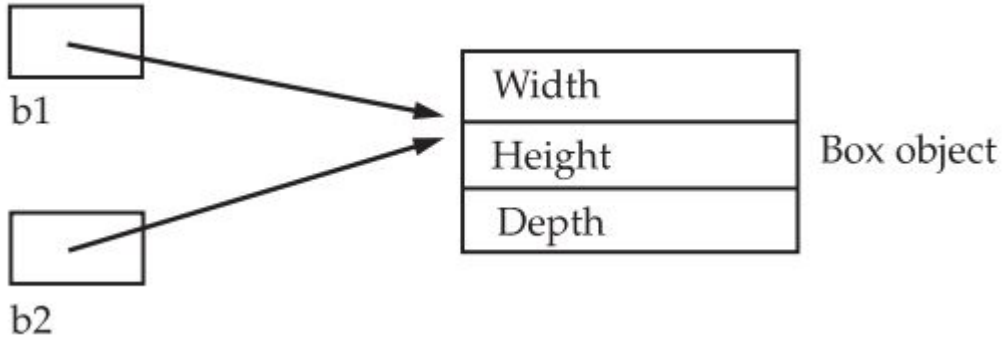
Assigning Object Reference Variables

```
Box b1 = new Box();
```

```
Box b2 = b1;
```

- It makes b2 refer to the same object as does b1.
- Thus, any changes made to the object through b2 will affect the object to which b1 is referring, since they are the same object.

Assigning Object Reference Variables (Cont.)



Assigning Object Reference Variables (Cont.)

```
Box b1 = new Box();  
Box b2 = b1;  
// ...  
b1 = null;
```

Here, **b1** has been set to **null**, but **b2** still points to the original object.

Assigning one object reference variable to another object reference variable doesn't mean creating a copy of the object, it only means making a copy of the reference.

Methods

```
type name(parameter-list) {  
    // body of method  
}
```

- Here, *type* specifies the type of data returned by the method. This can be any valid type, including class types that you create.
- If the method does not return a value, its return type must be `void`.
- The name of the method is specified by *name*.
- The *parameter-list* is a sequence of type and identifier pairs separated by commas.

Adding a Method to the Box Class

```
package com.company;

public class Box {
    double width;
    double height;
    double depth;

    // display volume of a box
    void volume() {
        System.out.print("Volume is ");
        System.out.println(width * height * depth);
    }
}
```

```
package com.company;

public class BoxDemo {
    public static void main(String args[]) {
        Box mybox1 = new Box();
        Box mybox2 = new Box();
        // assign values to mybox1's instance variables
        mybox1.width = 10;
        mybox1.height = 20;
        mybox1.depth = 15;
        /* assign different values to mybox2's
        instance variables */
        mybox2.width = 3;
        mybox2.height = 6;
        mybox2.depth = 9;
        // display volume of first box
        mybox1.volume();
        // display volume of second box
        mybox2.volume();
    }
}
```

Returning Value

```
package com.company;

public class Box {
    double width;
    double height;
    double depth;

    // compute and return volume
    double volume() {
        return width * height * depth;
    }
}
```

```
class BoxDemo4 {  
    public static void main(String args[]) {  
        Box mybox1 = new Box();  
        Box mybox2 = new Box();  
        double vol;  
  
        // assign values to mybox1's instance variab  
        mybox1.width = 10;  
        mybox1.height = 20;  
        mybox1.depth = 15;  
  
        /* assign different values to mybox2's  
           instance variables */  
        mybox2.width = 3;  
        mybox2.height = 6;  
        mybox2.depth = 9;  
  
        // get volume of first box  
        vol = mybox1.volume();  
        System.out.println("Volume is " + vol);  
  
        // get volume of second box  
        vol = mybox2.volume();  
        System.out.println("Volume is " + vol);  
    }  
}
```

Returning Value (Cont.)

- The type of data returned by a method must be compatible with the return type specified by the method. For example, if the return type of some method is boolean, you could not return an integer.
- The variable receiving the value returned by a method (such as vol, in this case) must also be compatible with the return type specified for the method.

Method

```
int square(int i){  
    return i * i;  
}
```

- A parameter is a variable defined by a method that receives a value when the method is called. For example, in `square( )`, `i` is a parameter.
- An argument is a value that is passed to a method when it is invoked. For example, `square(100)` passes 100 as an argument.

Constructors

- A constructor initializes an object immediately upon creation.
- It has the same name as the class in which it resides and is syntactically similar to a method.
- Once defined, the constructor is automatically called when the object is created, before the new operator completes.
- It is the constructor's job to initialize the internal state of an object so that the code creating an instance will have a fully initialized, usable object immediately.

```
/* Here, Box uses a constructor to initialize the
   dimensions of a box.
*/
class Box {
    double width;
    double height;
    double depth;

    // This is the constructor for Box.
    Box() {
        System.out.println("Constructing Box");
        width = 10;
        height = 10;
        depth = 10;
    }

    // compute and return volume
    double volume() {
        return width * height * depth;
    }
}
```

Parameterized Constructors

```
Box(double w, double h, double d) {  
    width = w;  
    height = h;  
    depth = d;  
}
```

```
Box mybox1 = new Box( w: 10, h: 20, d: 15);  
Box mybox2 = new Box( w: 3, h: 6, d: 9);
```

The **this** Keyword

- **this** can be used inside any method to refer to the current object.
- That is, **this** is always a reference to the object on which the method was invoked.

The **this** Keyword - Instance Variable Hiding

- When a local variable has the same name as an instance variable, the local variable hides the instance variable.
- **this** resolves any namespace collisions that might occur between instance variables and local variables.

```
Box(double width, double height, double depth) {  
    this.width = width;  
    this.height = height;  
    this.depth = depth;  
}
```

Garbage Collection

- In some languages, such as C++, dynamically allocated objects must be manually released by use of a *delete* operator.
- Java handles deallocation automatically. The technique that accomplishes this is called garbage collection.
- It works like this: when no references to an object exist, that object is assumed to be no longer needed, and the memory occupied by the object can be reclaimed.

The finalize() Method

- By using finalization, we can define specific actions that will occur when an object is just about to be reclaimed by the garbage collector.
- To add a finalizer to a class, we define the finalize() method.
- The Java run time calls that method whenever it is about to recycle an object of that class.
- Inside the finalize() method, we need to specify those actions that must be performed before an object is destroyed.
- Right before an asset is freed, the Java run time calls the finalize() method on the object.

The finalize() Method (Cont.)

The finalize() method has this general form:

```
protected void finalize( )  
{  
    // finalization code here  
}
```

- finalize() is only called just prior to garbage collection. We cannot know when—or even if—finalize() will be executed. Therefore, the program should provide other means of releasing system resources, etc., used by the object. It must not rely on finalize() for normal program operation.

Class & Encapsulation

- The class is the mechanism by which encapsulation is achieved in Java.
- By creating a class, we are creating a new data type that defines both the nature of the data being manipulated and the routines used to manipulate it.
- We can use the class through its methods without having to worry about the details of its implementation or how the data is actually managed within the class.
- In a sense, a class is like a “data engine.” No knowledge of what goes on inside the engine is required to use the engine through its controls.
- As long as your code uses the class through its methods, internal details can change without causing side effects outside the class.

```
class Stack {
    int stck[] = new int[10];
    int tos;

    // Initialize top-of-stack
    Stack() {
        tos = -1;
    }

    // Push an item onto the stack
    void push(int item) {
        if(tos==9)
            System.out.println("Stack is full.");
        else
            stck[++tos] = item;
    }

    // Pop an item from the stack
    int pop() {
        if(tos < 0) {
            System.out.println("Stack underflow.");
            return 0;
        }
        else
            return stck[tos--];
    }
}
```

```
class TestStack {  
    public static void main(String args[]) {  
        Stack mystack1 = new Stack();  
        Stack mystack2 = new Stack();  
  
        // push some numbers onto the stack  
        for(int i=0; i<10; i++) mystack1.push(i);  
        for(int i=10; i<20; i++) mystack2.push(i);  
  
        // pop those numbers off the stack  
        System.out.println("Stack in mystack1:");  
        for(int i=0; i<10; i++)  
            System.out.println(mystack1.pop());  
  
        System.out.println("Stack in mystack2:");  
        for(int i=0; i<10; i++)  
            System.out.println(mystack2.pop());  
    }  
}
```

Overloading Methods

- In Java, it is possible to define two or more methods within the same class that share the same name, as long as their parameter declarations are different. When this is the case, the methods are said to be overloaded, and the process is referred to as method overloading.
- Method overloading is one of the ways that Java supports **polymorphism**.

```
void test() {  
    System.out.println("No parameters");  
}  
  
// Overload test for one integer parameter.  
void test(int a) {  
    System.out.println("a: " + a);  
}  
  
// Overload test for two integer parameters.  
void test(int a, int b) {  
    System.out.println("a and b: " + a + " " + b);  
}  
  
// Overload test for a double parameter  
double test(double a) {  
    System.out.println("double a: " + a);  
    return a * a;  
}
```

```
class Overload {  
    public static void main(String args[]) {  
        OverloadDemo ob = new OverloadDemo();  
        double result;  
        // call all versions of test()  
        ob.test();  
        ob.test( a: 10);  
        ob.test( a: 10, b: 20);  
        result = ob.test( a: 123.25);  
        System.out.println("Result of ob.test(123.25): " + result);  
    }  
}
```

Constructor Overloading

```
class Box {
    double width;
    double height;
    double depth;

    // constructor used when all dimensions specified
    Box(double w, double h, double d) {
        width = w;
        height = h;
        depth = d;
    }

    // constructor used when no dimensions specified
    Box() {
        width = -1; // use -1 to indicate
        height = -1; // an uninitialized
        depth = -1; // box
    }

    // constructor used when cube is created
    Box(double len) {
        width = height = depth = len;
    }

    // compute and return volume
    double volume() {
        return width * height * depth;
    }
}
```

Returning Objects

```
class Test {
    int a;

    Test(int i) {
        a = i;
    }

    Test incrByTen() {
        Test temp = new Test(a + 10);
        return temp;
    }
}

class RetOb {
    public static void main(String args[]) {
        Test ob1 = new Test(2);
        Test ob2;
        ob2 = ob1.incrByTen();
        System.out.println("ob1.a: " + ob1.a);
        System.out.println("ob2.a: " + ob2.a);
        ob2 = ob2.incrByTen();
        System.out.println("ob2.a after second increase: " + ob2.a);
    }
}
```

Recursion

```
// A simple example of recursion.
class Factorial {
    // this is a recursive method
    int fact(int n) {
        int result;

        if(n==1) return 1;
        result = fact(n-1) * n;
        return result;
    }
}

class Recursion {
    public static void main(String args[]) {
        Factorial f = new Factorial();

        System.out.println("Factorial of 3 is " + f.fact(3));
        System.out.println("Factorial of 4 is " + f.fact(4));
        System.out.println("Factorial of 5 is " + f.fact(5));
    }
}
```

Access Control

- Encapsulation provides another important attribute: access control.
- Through encapsulation, we can control what parts of a program can access the members of a class.
- By controlling access, misuse can be prevented.
- How a member can be accessed is determined by the access modifier attached to its declaration.
- Java's access modifiers are public, private, and protected. Java also defines a default access level.

Access Control (Cont.)

- **public** member can be accessed by any other code.
- **private** member can only be accessed by other members of its class.
- When no access modifier is used, then by default the member of a class is public within its own package, but cannot be accessed outside of its package.
- An access modifier precedes the rest of a member's type specification.

Access Control

```
class Test {  
    int a; // default access  
    public int b; // public access  
    private int c; // private access  
  
    // methods to access c  
    void setc(int i) { c = i; }  
  
    int getc() { return c; }  
}  
  
class AccessTest {  
    public static void main(String args[]) {  
        Test ob = new Test();  
        // These are OK, a and b may be accessed directly  
        ob.a = 10;  
        ob.b = 20;  
        // This is not OK and will cause an error  
        // ob.c = 100; // Error!  
        // You must access c through its methods  
        ob.setc(100); // OK  
        System.out.println("a, b, and c: " + ob.a + " " +  
            ob.b + " " + ob.getc());  
    }  
}
```

Static

- When a member is declared static, it can be accessed before any objects of its class are created, and without reference to any object.
- Both methods and variables can be static.
- `main( )` is declared as static because it must be called before any objects exist.
- Instance variables declared as static are, essentially, global variables.
- When objects of its class are declared, no copy of a static variable is made. Instead, all instances of the class share the same static variable.

Static (Cont.)

Methods declared as static have several restrictions:

- They can only directly call other static methods.
- They can only directly access static data.
- They cannot refer to **this** or **super** in any way.

Static

```
class StatClass {  
    static int a;  
    int b;  
}  
  
public class StaticDemo {  
    int c;  
  
    static void display(int c) {  
        StatClass.a = 5;  
        StatClass.b = 10;  
        this.c = c;  
        StatClass statClass = new StatClass();  
        statClass.b = 10;  
    }  
  
    void show() {  
    }  
  
    public static void main(String[] args) {  
        display(c: 8);  
        StaticDemo staticDemo = new StaticDemo();  
        staticDemo.show();  
        show();  
    }  
}
```

Final

- Declaring a field as final prevents its contents from being modified, making it, essentially, a constant.
- It is a common coding convention to choose all uppercase identifiers for final fields. For example, `final int FILE_NEW = 1;`
- In addition to fields, both method parameters and local variables can be declared final.
- Declaring a parameter final prevents it from being changed within the method.
- Declaring a local variable final prevents it from being assigned a value more than once.

Final

```
public class finalDemo {  
    final int A = 5;  
    int b= 10;  
  
    void test (final int c, int d) {  
        //      A = 10; //error  
        b = 15;  
        //      C = 20; //error  
        d=20;  
    }  
    public static void main(String[] args) {  
    }  
}
```

Arrays

- The number of elements that an array can hold—is found in its **length** instance variable.
- The value of length has nothing to do with the number of elements that are actually in use. It only reflects the number of elements that the array is designed to hold.

Arrays (Cont.)

```
// This program demonstrates the length array member.
class Length {
    public static void main(String args[]) {
        int a1[] = new int[10];
        int a2[] = {3, 5, 7, 1, 8, 99, 44, -10};
        int a3[] = {4, 3, 2, 1};

        System.out.println("length of a1 is " + a1.length);
        System.out.println("length of a2 is " + a2.length);
        System.out.println("length of a3 is " + a3.length);
    }
}
```

Nested and Inner Classes

- A class within another class is known as nested class.
- A nested class has access to the members, including private members, of the class in which it is nested.
- However, the **enclosing** class does not have access to the members of the nested class.
- There are two types of nested classes: static and non-static.
- A static nested class must access the non-static members of its enclosing class through an object. That is, it cannot refer to non-static members of its enclosing class directly.

Nested and Inner Classes (Cont.)

- The most important type of nested class is the inner class.
- An inner class is a non-static nested class.
- It has access to all of the variables and methods of its outer class and may refer to them directly in the same way that other non-static members of the outer class do.

Nested and Inner Classes

```
class Outer {
    int outer_x = 100;
    void test() {
        //      System.out.println(inner_x); //error
        Inner inner = new Inner();
        inner.display();
    }
    // this is an inner class
    class Inner {
        int inner_x = 50;
        void display() {
            System.out.println("display: outer_x = " + outer_x);
        }
    }

    static class InnStat {
        void show() {
            //      System.out.println(outer_x); //error
        }
    }
}

class InnerClassDemo {
    public static void main(String args[]) {
        Outer outer = new Outer();
        outer.test();
    }
}
```

Nested and Inner Classes

```
// Define an inner class within a for loop.
class Outer {
    int outer_x = 100;

    void test() {
        for(int i=0; i<10; i++) {
            class Inner {
                void display() {
                    System.out.println("display: outer_x = " + outer_x);
                }
            }
            Inner inner = new Inner();
            inner.display();
        }
    }
}

class InnerClassDemo {
    public static void main(String args[]) {
        Outer outer = new Outer();
        outer.test();
    }
}
```

String

- Every string is actually an object of type String. Even string constants are actually String objects.
- String are immutable. Once a String object is created, its contents cannot be altered.
- If we need to change a string, we can create a new one that contains the modifications.
- Java defines peer classes of String, called **StringBuffer** and **StringBuilder**, which allow strings to be altered, so all of the normal string manipulations are still available in Java.

String (Cont.)

- `String myString = "this is a test";`
- `String myString = "I" + " like " + "Java.";`
- Two strings for equality can be tested by using **`equals( )`**.
- The length of a string can be obtained by calling the **`length( )`** method.
- The character at a specified index within a string can be obtained by calling **`charAt( )`**.
- `==` checks if both objects point to the same memory location whereas **`.equals()`** evaluates to the comparison of values in the objects.

Command-Line Arguments

- Passing information into a program when we run it can be accomplished by passing command-line arguments to `main( )`.
- A command-line argument is the information that directly follows the program's name on the command line when it is executed.
- Command-line arguments are stored as strings in a `String` array passed to the `args` parameter of `main( )`.
- The first command-line argument is stored at `args[0]`, the second at `args[1]`, and so on.

Varargs: Variable-Length Arguments

- A method that takes a variable number of arguments is called a variable-arity method, or simply a varargs method.
- A variable-length argument is specified by three periods (...).
For example, `static void vaTest(int ... v) { }`
- The above syntax tells the compiler that `vaTest( )` can be called with zero or more arguments. As a result, `v` is implicitly declared as an array of type `int[ ]`. Thus, inside `vaTest( )`, `v` is accessed using the normal array syntax.

Varargs: Variable-Length Arguments (Cont.)

- A method can have “normal” parameters along with a variable-length parameter.
- The varargs parameter must be last.
- `int doIt(int a, int b, double c, int ... vals) { }`
- `int doIt(int a, int b, double c, int ... vals, boolean stopFlag) { //`
Error!

Variable-Length Arguments

```
// Demonstrate variable-length arguments.
class VarArgs {

    // vaTest() now uses a vararg.
    static void vaTest(int ... v) {
        System.out.print("Number of args: " + v.length +
                          " Contents: ");

        for(int x : v)
            System.out.print(x + " ");

        System.out.println();
    }

    public static void main(String args[])
    {
        // Notice how vaTest() can be called with a
        // variable number of arguments.
        vaTest(10);           // 1 arg
        vaTest(1, 2, 3);      // 3 args
        vaTest();             // no args
    }
}
```

Overloading Vararg Methods

```
// Varargs and overloading.
class VarArgs3 {

    static void vaTest(int ... v) {
        System.out.print("vaTest(int ...): " +
            "Number of args: " + v.length +
            " Contents: ");

        for(int x : v)
            System.out.print(x + " ");

        System.out.println();
    }

    static void vaTest(boolean ... v) {
        System.out.print("vaTest(boolean ...) " +
            "Number of args: " + v.length +
            " Contents: ");

        for(boolean x : v)
            System.out.print(x + " ");

        System.out.println();
    }
}
```

Varargs and Ambiguity

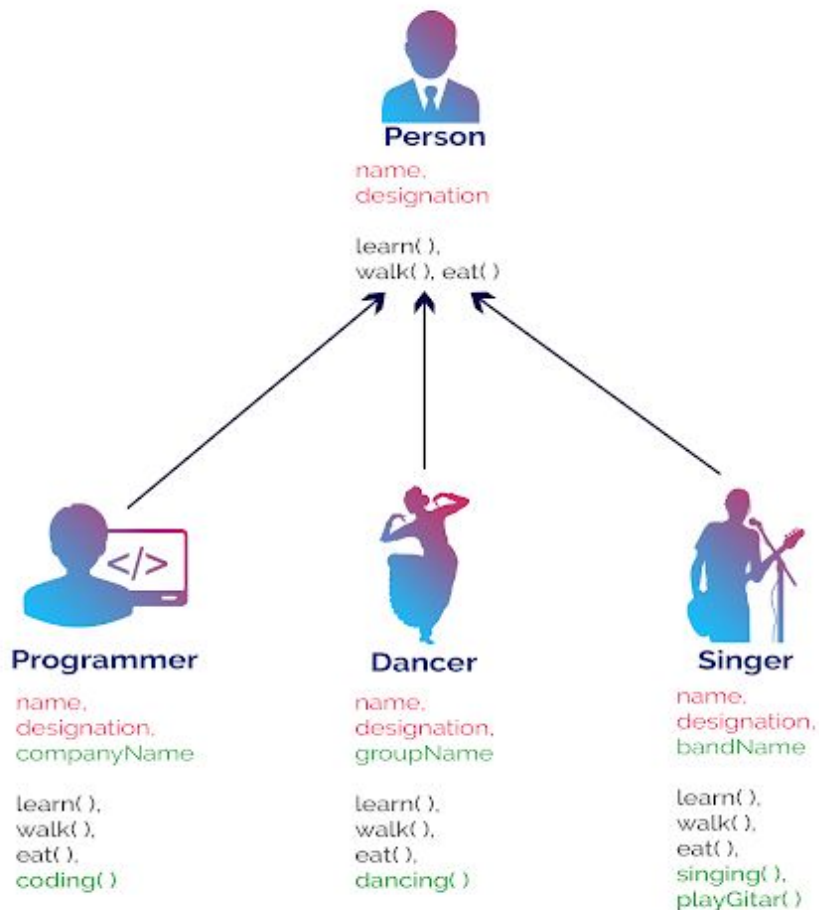
```
class VarArgs4 {  
    static void vaTest(int... v) {  
        System.out.print("vaTest(int ...): " +  
            "Number of args: " + v.length +  
            " Contents: ");  
        for (int x : v)  
            System.out.print(x + " ");  
        System.out.println();  
    }  
  
    static void vaTest(boolean... v) {  
        System.out.print("vaTest(boolean ...) " +  
            "Number of args: " + v.length +  
            " Contents: ");  
        for (boolean x : v)  
            System.out.print(x + " ");  
        System.out.println();  
    }  
  
    public static void main(String args[]) {  
        vaTest( ...v: 1, 2, 3); // OK  
        vaTest( ...v: true, false, false); // OK  
        vaTest(); // Error: Ambiguous!  
    }  
}
```

Terms

Runtime is the final phase of the program life cycle in which the machine executes the program's code.

Inheritance

Inheritance



Inheritance

- Inheritance allows the creation of hierarchical classifications.
- Using inheritance, you can create a general class that defines traits common to a set of related items.
- A class that is inherited is called a **superclass**. The class that does the inheriting is called a **subclass**.

Inheritance (Cont.)

- A subclass is a specialized version of a superclass. It inherits all of the members defined by the superclass and adds its own, unique elements.
- **extends** keyword is used to create a subclass of A.
- Java does not support the inheritance of multiple superclasses into a single subclass.

Inheritance (Cont.)

- No class can be a superclass of itself.
- A subclass includes all of the members of its superclass except private members.
- Each subclass can precisely tailor its own classification.

Inheritance

```
class SupClass {  
    int i = 10;  
    private int k = 15;  
}  
  
class SubClass extends SupClass{  
    int j = 5;  
    void display () {  
        System.out.println(i+j); // i is declared in its super class  
        // System.out.println(i+j+k); //error because private var k is not accessible  
    }  
}  
  
public class InheritanceDemo {  
    public static void main(String[] args) {  
        SubClass subClass = new SubClass();  
        subClass.display();  
    }  
}
```

A Superclass Variable Can Reference a Subclass Object

```
class Box {  
    double height;  
}  
  
class BoxWeight extends Box{  
    double weight;  
}  
  
public class RefDemo {  
    public static void main(String[] args) {  
        Box box = new Box();  
        BoxWeight boxWeight = new BoxWeight();  
  
        box = boxWeight; // Assign BoxWeight Reference to Box Reference  
        // boxWeight = box; // Can't assign Box Reference to BoxWeight Reference  
  
        box.height = 10;  
        // box.weight = 10; //error  
    }  
}
```

Super

- The **super** keyword in java is a reference variable that is used to refer parent class objects.
- A subclass can refer to its immediate superclass by using the keyword **super**.
- **super** has two general forms.
- The first form calls the superclass' constructor.
- The second is used to access a member of the superclass that has been hidden by a member of a subclass.

Super(Cont.)

- `super( )` must always be the first statement executed inside a subclass constructor.
- If a constructor of subclass does not invoke a superclass constructor, the Java compiler automatically inserts a call to the no-argument constructor of the superclass.
- If a superclass constructor requires parameters, then all subclasses must pass those parameters “up the line.”

```
class SuperClass {  
    int a;  
  
    SuperClass(int b) {  
        a = b;  
    }  
}  
  
class SubClass extends SuperClass {  
    SubClass(int b) {  
        super(b);  
    }  
  
    void display() {  
        System.out.println(a);  
    }  
}
```

```
public class SuperDemo {  
    public static void main(String[] args) {  
        int num = 10;  
        SubClass subClass = new SubClass(num);  
        subClass.display();  
    }  
}
```

Use of **Super** With Constructors

```

class SpClass {
    int a = 5;
    void printMsg() {
        System.out.println("Super Class");
    }
}

class SbClass extends SpClass {
    int a = 10;
    void display() {
        System.out.println(a); //prints 10
        System.out.println(super.a); //prints 5
    }
    void printMsg() {
        super.printMsg(); //prints "Super Class"
        System.out.println("Sub Class");
    }
}

```

```

public class SuperDemo {
    public static void main(String[] args) {
        SbClass sbClass = new SbClass();
        sbClass.display();
        sbClass.printMsg();
    }
}

```

Use of **super** to access
Superclass hidden members

```
class A {  
    int a;  
    A() {  
        System.out.println("Inside A's constructor.");  
    }  
}  
  
class B extends A {  
    int b;  
    B() {  
        System.out.println("Inside B's constructor.");  
    }  
}  
  
class C extends B {  
    int c;  
    C() {  
        System.out.println("Inside C's constructor.");  
    }  
}
```

```
public class MultilevelHierarchy {  
    public static void main(String[] args) {  
        C c = new C();  
        c.a = 5;  
        c.b = 10;  
        c.c = 15;  
    }  
}
```

Multilevel Hierarchy

Method Overriding

- Subclass (child class) has the same method as declared in the parent class.
- Same method name and type signature.
- Overridden method called from subclass, always refer to the version of that method defined by the subclass.

```

class Aa{
    int i, j;
    Aa(int a, int b) {
        i = a;
        j = b;
    }
    // display i and j
    void show() {
        System.out.println("i and j: " + i + " " + j);
    }
}

class Bb extends Aa{
    int k;
    Bb(int a, int b, int c) {
        super(a, b);
        k = c;
    }
    // display k - this overrides show() in A
    void show() {
        super.show();
        System.out.println("k: " + k);
    }
}

```

```

public class MethodOverriding {
    public static void main(String args[]) {
        Bb subOb = new Bb( a: 1, b: 2, c: 3);
        subOb.show(); // this calls show() in B
    }
}

```

Method Overriding

Dynamic Method Dispatch

- Dynamic method dispatch is the mechanism by which a call to an overridden method is resolved at run time, rather than compile time.
- Method overriding forms the basis for one of Java's most powerful concepts: dynamic method dispatch.
- Dynamic method dispatch is important because this is how Java implements run-time polymorphism.

Dynamic Method Dispatch (Cont.)

- When an overridden method is called through a superclass reference, Java determines which version of that method to execute based upon the type of the object being referred to at the time the call occurs.
- It is the type of the object being referred to determines which version of an overridden method will be executed.

```

class A {
    void callme() {
        System.out.println("Inside A's callme method");
    }
}

class B extends A {
    // override callme()
    void callme() {
        System.out.println("Inside B's callme method");
    }
}

class C extends A {
    // override callme()
    void callme() {
        System.out.println("Inside C's callme method");
    }
}

```

```

class Dispatch {
    public static void main(String args[]) {
        A a = new A(); // object of type A
        B b = new B(); // object of type B
        C c = new C(); // object of type C
        A r; // obtain a reference of type A
        r = a; // r refers to an A object
        r.callme(); // calls A's version of callme
        r = b; // r refers to a B object
        r.callme(); // calls B's version of callme
        r = c; // r refers to a C object
        r.callme(); // calls C's version of callme
    }
}

```

Dynamic Method Dispatch

Why Overridden Methods

- Overridden methods allow Java to support run-time polymorphism.
- Polymorphism is essential to object-oriented programming for one reason.
- It allows a general class to specify methods that will be common to all of its derivatives.
- It allows subclasses to define the specific implementation of some or all of those methods.

Why Overridden Methods(Cont.)

- Overridden methods are another way that Java implements the “one interface, multiple methods” aspect of polymorphism.
- Dynamic, run-time polymorphism is one of the most powerful mechanisms that object oriented design brings to bear on code reuse and robustness.

Abstract Class

- Abstraction is a process of hiding the implementation details and showing only functionality to the user.
- It focuses on what the object does instead of how it does it.
- Two ways to achieve abstraction in java: Abstract class and Interface.
- An abstract class must be declared with an **abstract** keyword.

Abstract Class (Cont.)

- There can be no objects of an abstract class.
- They can be used to create object references.
- An abstract class cannot be directly instantiated with the new operator.
- We cannot declare abstract constructors, or abstract static methods.
- It can have constructors and static methods.

Abstract Class (Cont.)

- Any subclass of an abstract class must either implement all of the abstract methods in the superclass, or be declared abstract itself.
- Abstract method can only be used in an abstract class, and it does not have a body.
- An abstract class can have both abstract and regular methods.

```

abstract class A {

    abstract void callme();

    void callmetoo() {
        System.out.println("concrete");
    }
}

class B extends A {
    void callme() {
        System.out.println("B's implementation of callme.");
    }
}

```

```

class AbstractDemo {
    public static void main(String args[]) {
        //      A a = new A(); //error
        A a;
        B b = new B();
        b.callme();
        b.callmetoo();

        a = b;
        a.callme();
    }
}

```

Abstract Class

Using final to Prevent Overriding

Methods declared as final cannot be overridden.

```
class A {  
    final void meth() {  
        System.out.println("This is a final method.");  
    }  
}  
  
class B extends A {  
    void meth() { // ERROR! Can't override.  
        System.out.println("Illegal!");  
    }  
}
```

Using final to Prevent Inheritance

- Declaring a class as final implicitly declare all of its methods as final, too.
- A final class can't be inherited.
- It is illegal to declare a class as both abstract and final.

Using final to Prevent Inheritance

```
final class A {  
    //...  
}
```

```
// The following class is illegal.  
class B extends A { // ERROR! Can't subclass A  
    //...  
}
```

Object Class

- Object is a superclass of all other classes.
- A reference variable of type Object can refer to an object of any other class.
- A variable of type Object can also refer to any array.

Methods Defined by Object

Method	Purpose
Object clone()	Creates a new object that is the same as the object being cloned.
boolean equals(Object <i>object</i>)	Determines whether one object is equal to another.
void finalize()	Called before an unused object is recycled.
Class<?> getClass()	Obtains the class of an object at run time.
int hashCode()	Returns the hash code associated with the invoking object.
void notify()	Resumes execution of a thread waiting on the invoking object.
void notifyAll()	Resumes execution of all threads waiting on the invoking object.
String toString()	Returns a string that describes the object.
void wait() void wait(long <i>milliseconds</i>) void wait(long <i>milliseconds</i> , int <i>nanoseconds</i>)	Waits on another thread of execution.

Packages & Interfaces

Importing Packages (Cont.)

- Only the items declared as public within the imported package will be available to non-subclasses in the importing code.

Packages

- Packages are containers for classes.
- The package is both a naming and a visibility control mechanism.
- Classes can be defined inside a package that are not accessible by code outside that package.
- Class members can be defined in a way that are exposed only to other members of the same package.

Packages

- The package statement defines a name space in which classes are stored.
- Omitting the package statement, put classes into the default package, which has no name.
- While the default package is fine for short, sample programs, it is inadequate for real applications.

- General form of the package statement: **package** MyPackage;
- Java uses file system directories to store packages.
- The .class files for any classes of MyPackage must be stored in a directory called MyPackage.
- Case is significant.
- The directory name must match the package name exactly.
- The package statement simply specifies to which package the classes defined in a file belong.

- We can create a hierarchy of packages. For example,
package pkg1[.pkg2[.pkg3]];
- A package hierarchy must be reflected in the file system of Java development system.
- **package** java.awt.image; needs to be stored in java\awt\image in a Windows environment.

Finding Packages and CLASSPATH

- By default, the Java run-time system uses the current working directory as its starting point.
 - If the package is in a subdirectory of the current directory, it will be found.

Finding Packages and CLASSPATH (Cont.)

- A directory path or paths can be specified by setting the CLASSPATH environmental variable.
- We can use the -classpath option with java and javac to specify the path to classes.

Importing Packages

- Java includes the **import** statement to bring certain classes, or entire packages, into visibility.
- Once imported, a class can be referred to directly, using only its name.
- The **import** statement is a convenience to the programmer.
- General form of the import statement: **import** pkg1
[.pkg2].(classname | *);

Importing Packages (Cont.)

- `import java.util.Date;`
- `import java.io.*;`
- All of the standard Java classes included with Java are stored in a package called `java`.
- The basic language functions are stored in a package inside of the `java` package called `java.lang`.

Importing Packages (Cont.)

- Every package or class that is used needs to be imported.
- Since Java is useless without much of the functionality in `java.lang`, it is implicitly imported by the compiler for all programs.

Optional **import** statement

```
import java.util.*;  
class MyDate extends Date {  
}
```

The same example without the **import** statement looks like this:

```
class MyDate extends java.util.Date {  
}
```

Interfaces

- Interface can specify what a class must do, but not how it does it.
- They lack instance variables.
- Their methods are declared without any body.
- Any number of classes can implement an interface.
- One class can implement any number of interfaces.

Interface(Cont.)

- Prior to JDK 8, an interface could define only “what,” but not “how.”
- Beginning with JDK 8, it is possible to add a default implementation to an interface method.
- Variables can be declared inside of interface declarations.
- Variables are implicitly final and static.
- Variables must also be initialized.

Interface(Cont.)

- Variables cannot be changed by the implementing class.
- All methods and variables are implicitly public.
- The methods that implement an interface must be declared public.

```

class InterfaceImp implements InterfaceDemoA, InterfaceDemoB {
    public static void main(String[] args) {
        //      a=3; //error
        System.out.println(a);
    }

    @Override
    public void display() {
        System.out.println("Hello");
    }

    void nonIfaceMeth() {
        System.out.println("Classes that implement interfaces
            + "may also define other members, too.");
    }

    @Override
    public void show() {
        System.out.println("Hi");
    }
}

```

```

interface InterfaceDemoA {
    //      int a; //error
    int a = 5;
    void display();
}

interface InterfaceDemoB {
    void show();
}

```

Interface Implementation

Accessing Implementations Through Interface References

- Declaring variables as object references using an interface type.
- Any instance of any class that implements the declared interface can be referred to by such a variable.
- The method to be executed is looked up dynamically at run time.

Interface References

```
interface Callback {  
    void callback(int param);  
}  
  
class Client implements Callback {  
    // Implement Callback's interface  
    public void callback(int p) {  
        System.out.println("callback called with " + p);  
    }  
}  
  
public class TestIface {  
    public static void main(String args[]) {  
        Callback c = new Client();  
        c.callback( param: 42);  
    }  
}
```

Interface References

```
// Another implementation of Callback.
class AnotherClient implements Callback {
    // Implement Callback's interface
    public void callback(int p) {
        System.out.println("Another version of callback");
        System.out.println("p squared is " + (p*p));
    }
}
```

Now, try the following class:

```
class TestIface2 {
    public static void main(String args[]) {
        Callback c = new Client();
        AnotherClient ob = new AnotherClient();

        c.callback(42);

        c = ob; // c now refers to AnotherClient object
        c.callback(42);
    }
}
```

```
interface Callback {  
    void callback(int param);  
}  
  
abstract class Incomplete implements Callback {  
    int a, b;  
  
    void show() {  
        System.out.println(a + " " + b);  
    }  
}
```

Partial Interface Implementation

Nested Interfaces

- An interface can be declared a member of a class or another interface. Such an interface is called a member interface or a nested interface.
- A nested interface can be declared as public, private, or protected.
- Top-level interface, must either be declared as public or use the default access level.

```
class AN {  
    // this is a nested interface  
    public interface NestedIF {  
        boolean isNotNegative(int x);  
    }  
}  
  
// B implements the nested interface.  
class BN implements AN.NestedIF {  
    public boolean isNotNegative(int x) {  
        return x < 0 ? false : true;  
    }  
}
```

```
class NestedIFDemo {  
    public static void main(String args[]) {  
        // use a nested interface reference  
        AN.NestedIF nif = new BN();  
        if (nif.isNotNegative(x: 10))  
            System.out.println("10 is not negative");  
        if (nif.isNotNegative(x: -12))  
            System.out.println("this won't be displayed");  
    }  
}
```

Nested Interfaces

```
interface Inf {  
    void display();  
}  
  
class InfImp1 implements Inf {  
  
    @Override  
    public void display() {  
        System.out.println("InfImp1");  
    }  
}  
  
class InfImp2 implements Inf {  
  
    @Override  
    public void display() {  
        System.out.println("InfImp2");  
    }  
}
```

```
public class ApplyInterfaceDemo {  
    public static void main(String[] args) {  
        Inf inf;  
        InfImp1 infImp1 = new InfImp1();  
        InfImp2 infImp2 = new InfImp2();  
  
        inf = infImp1;  
        inf.display();  
  
        inf = infImp2;  
        inf.display();  
    }  
}
```

Applying Interfaces

Variables in Interfaces

```
interface SharedConstants {  
    int NO = 0;  
    int YES = 1;  
}  
  
class Question implements SharedConstants {  
    Random rand = new Random();  
  
    int ask() {  
        int prob = (int) (100 * rand.nextDouble());  
        if (prob < 30)  
            return NO;  
        else return YES;  
    }  
}  
  
class AskMe implements SharedConstants {  
    public static void main(String args[]) {  
        Question q = new Question();  
        System.out.println(q.ask());  
        System.out.println(q.ask());  
    }  
}
```

Extended Interfaces

```
interface AIf {  
    void meth1();  
}  
  
interface BIf extends AIf {  
    void meth2();  
}  
  
class MyClass implements BIf {  
    public void meth1() {  
        System.out.println("Implement meth1().");  
    }  
  
    public void meth2() {  
        System.out.println("Implement meth2().");  
    }  
}
```

Default Interface Methods

- The release of JDK 8 added a new capability to interface called the default method.
- A default method lets us define a default implementation for an interface method.
- Provides a means by which interfaces could be expanded without breaking existing code.

```
interface MyIF {  
    int getNumber();  
    default String getString() {  
        return "Default String";  
    }  
}  
  
class MyIFImp implements MyIF {  
    public int getNumber() {  
        return 100;  
    }  
}
```

```
public class DefaultMethodDemo {  
    public static void main(String args[]) {  
        MyIFImp obj = new MyIFImp();  
        System.out.println(obj.getNumber());  
        System.out.println(obj.getString());  
    }  
}
```

Default Method

```
interface Alpha {  
    default void reset() {  
        System.out.println("Alpha");  
    }  
}  
  
interface Beta {  
    default void reset() {  
        System.out.println("Beta");  
    }  
}
```

```
public class MultipleInheritance implements Alpha, Beta {  
    public static void main(String[] args) {  
        MultipleInheritance multipleInheritance  
            = new MultipleInheritance();  
        multipleInheritance.reset();  
    }  
  
    public void reset() {  
        System.out.println("Main Reset");  
    }  
}
```

Multiple Inheritance Issues

```
interface Alpha {  
    default void reset() {  
        System.out.println("Alpha");  
    }  
}  
  
interface Beta extends Alpha{  
    default void reset() {  
        System.out.println("Beta");  
        // Alpha.super.reset();  
    }  
}
```

```
public class MultipleInheritance implements Beta{  
    public static void main(String[] args) {  
        MultipleInheritance multipleInheritance  
            = new MultipleInheritance();  
        multipleInheritance.reset();  
    }  
}
```

Multiple Inheritance Issues

```
interface MyIF {  
    int getNumber();  
    default String getString() {  
        return "Default String";  
    }  
    static int getDefaultNumber() {  
        return 0;  
    }  
}
```

```
public class InfStaticMethod implements MyIF{  
    public static void main(String[] args) {  
        System.out.println(MyIF.getDefaultNumber());  
    }  
  
    @Override  
    public int getNumber() {  
        return 0;  
    }  
  
    @Override  
    public String getString() {  
        return MyIF.super.getString();  
    }  
}
```

Static Methods in an Interface

Exception Handling

Exception

- An exception is an abnormal condition that arises in a code sequence at run time. An exception is a runtime error.
- In computer languages that do not support exception handling, errors must be checked and handled manually.
- Java's exception handling brings run-time error management into the object-oriented world.

Exception (Cont.)

- A Java exception is an object that describes an exceptional (that is, error) condition in a piece of code.
- When an exceptional condition arises, an object representing that exception is created and thrown in the method that caused the error.
- The exception is caught and processed.

Exception (Cont.)

- Exceptions can be generated by the Java run-time system.
- Exceptions can be manually generated by your code.
- Exceptions thrown by Java relate to fundamental errors that violate the rules of the Java language or the constraints of the Java execution environment.

Exception Handling

Java exception handling is managed via five keywords.

- Program statements that you want to monitor for exceptions are contained within a ***try*** block.
- If an exception occurs within the **try** block, it is thrown.
- Your code can catch this exception (using ***catch***).

Exception Handling (Cont.)

- To manually throw an exception, use the keyword *throw*.
- Any exception that is thrown out of a method must be specified as such by a *throws* clause.
- Any code that absolutely must be executed after a try block completes is put in a *finally* block.

Exception Types

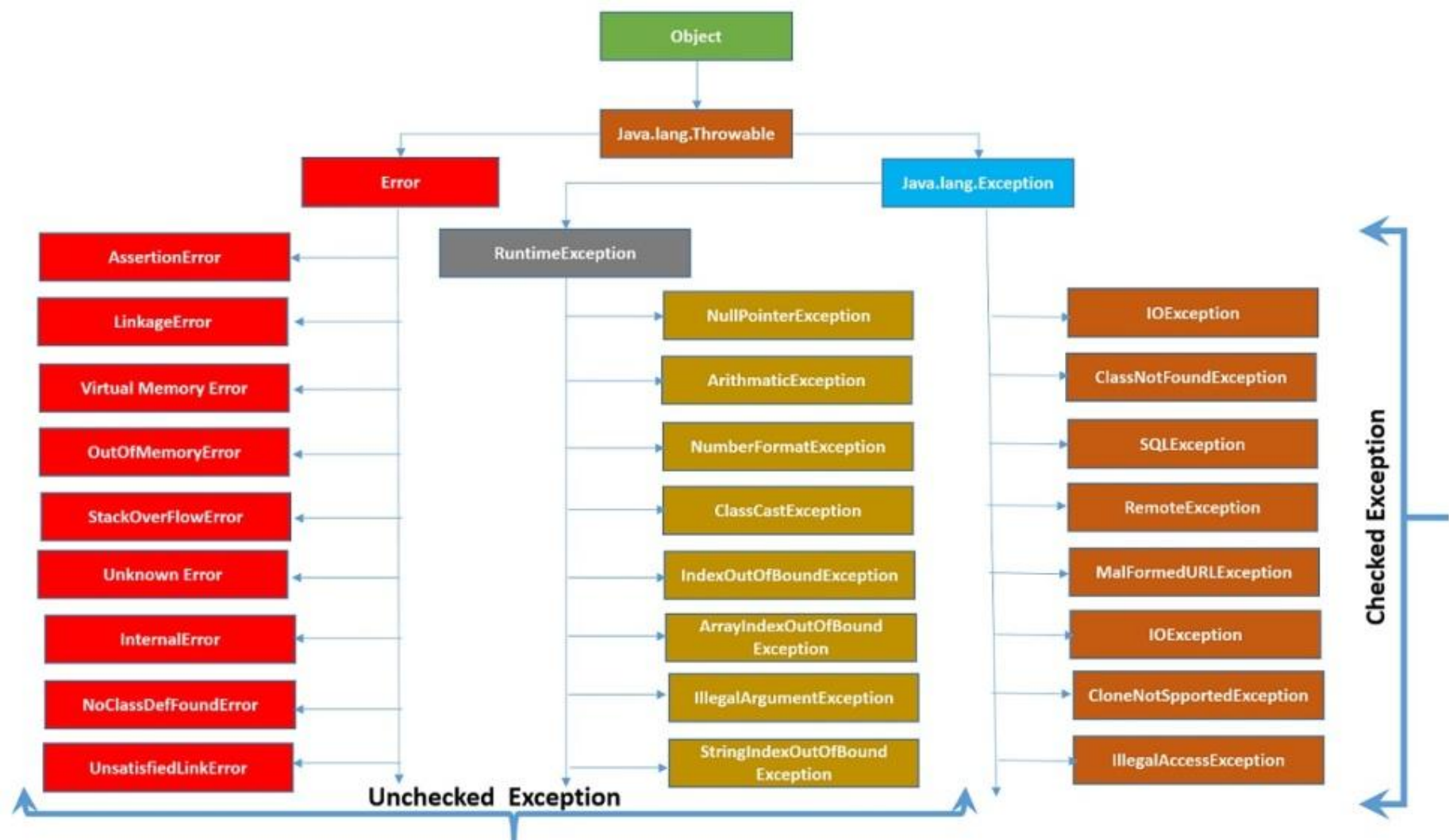
- All exception types are subclasses of the built-in class **Throwable**.
- **Exception** class is used for exceptional conditions that user programs should catch.
- **Exception** class is subclassed to create own custom exception types.

Exception Types (Cont.)

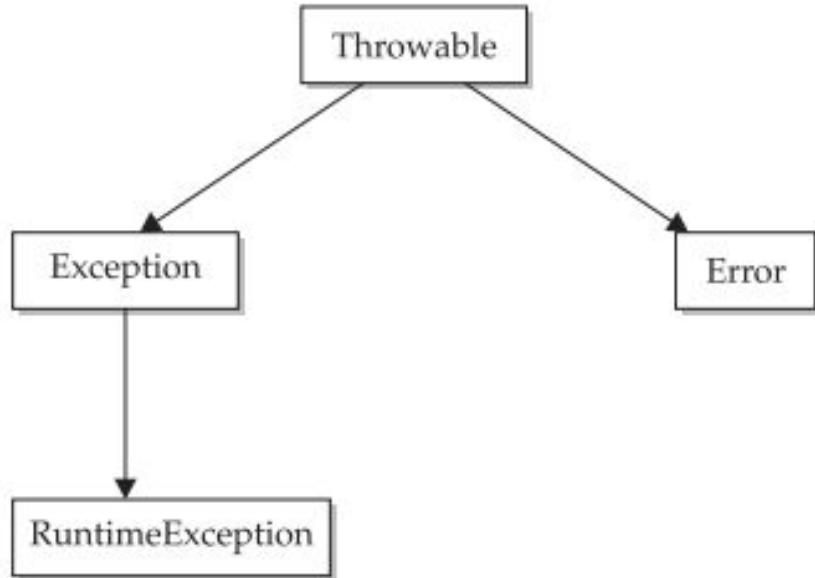
- Exceptions of type **RuntimeException** are automatically defined for the programs that we write.
- **RuntimeException** includes things such as division by zero and invalid array indexing.
- Exceptions of type **Error** are used by the Java run-time system to indicate errors having to do with the run-time environment, itself.

Exception Types (Cont.)

- **Stack overflow** is an example of type **Error**.



Exception Hierarchy



Uncaught Exceptions

```
class Exc0 {  
    public static void main(String args[]) {  
        int d = 0;  
        int a = 42 / d;  
    }  
}
```

```
java.lang.ArithmeticException: / by zero  
    at Exc0.main(Exc0.java:4)
```

Uncaught Exceptions (Cont.)

- When the Java run-time system detects the attempt to divide by zero, it constructs a new exception object and then throws this exception.
- This causes the execution of `Exc0` to stop.
- Because once an exception has been thrown, it must be caught by an exception handler and dealt with immediately.

Uncaught Exceptions (Cont.)

- In the previous example, the exception is caught by the default handler provided by the Java run-time System.
- Any exception that is not caught by your program will ultimately be processed by the default handler.
- The default handler displays a string describing the exception, prints a stack trace from the point at which the exception occurred, and terminates the program.

Uncaught Exceptions

```
class Excl {  
    static void subroutine() {  
        int d = 0;  
        int a = 10 / d;  
    }  
    public static void main(String args[]) {  
        Excl.subroutine();  
    }  
}
```

```
java.lang.ArithmeticException: / by zero  
    at Excl.subroutine(Excl.java:4)  
    at Excl.main(Excl.java:7)
```

Using try and catch

Handling an exception on own provides two benefits.

- First, it allows to fix the error.
- Second, it prevents the program from automatically terminating.

Using try and catch (Cont.)

```
public class Exc2 {  
    public static void main(String args[]) {  
        int d, a;  
        try { // monitor a block of code.  
            d = 0;  
            a = 42 / d;  
            System.out.println("This will not be printed.");  
        } catch (ArithmeticException e) { // catch divide-by-zero error  
            System.out.println("Division by zero.");  
        }  
        System.out.println("After catch statement.");  
    }  
}
```

Using try and catch (Cont.)

```
Division by zero.  
After catch statement.
```

Using try and catch (Cont.)

- A **catch** statement cannot catch an exception thrown by another **try** statement (except nested **try** statements).
- The statements that are protected by **try** must be surrounded by curly braces.
- The goal of most well-constructed **catch** clauses should be to resolve the exceptional condition and then continue on as if the error had never happened.

Displaying a Description of an Exception

- **Throwable** overrides the **toString()** method (defined by **Object**) so that it returns a string containing a description of the exception.
- This description can be displayed in a **println()** statement by simply passing the exception as an argument.
- The ability to display a description of an exception is valuable when experimenting with exceptions or debugging.

Displaying a Description of an Exception

```
catch (ArithmeticException e) {  
    System.out.println("Exception: " + e);  
    a = 0; // set a to zero and continue  
}
```

Multiple catch Clauses

- In some cases, more than one exception could be raised by a single piece of code.
- To handle this type of situation, two or more **catch** clauses can be specified.
- Each catches a different type of exception.

Multiple catch Clauses (Cont.)

- When an exception is thrown, each **catch** statement is inspected in order, and the first one whose type matches that of the exception is executed.
- After one **catch** statement executes, the others are bypassed, and execution continues after the **try/catch** block.

Multiple catch Clauses (Cont.)

```
class MultipleCatches {  
    public static void main(String args[]) {  
        try {  
            int a = args.length;  
            System.out.println("a = " + a);  
            int b = 42 / a;  
            int c[] = {1};  
            c[42] = 99;  
        } catch (ArithmeticException e) {  
            System.out.println("Divide by 0: " + e);  
        } catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Array index oob: " + e);  
        }  
        System.out.println("After try/catch blocks.");  
    }  
}
```

Multiple catch Clauses (Cont.)

- While using multiple catch statements, exception subclasses must come before any of their superclasses.
- Because a catch statement that uses a superclass will catch exceptions of that type plus any of its subclasses.
- Thus, a subclass would never be reached if it came after its superclass.
- In Java, unreachable code is an error.

Multiple catch Clauses (Cont.)

```
class SuperSubCatch {  
    public static void main(String args[]) {  
        try {  
            int a = 0;  
            int b = 42 / a;  
        } catch (Exception e) {  
            System.out.println("Generic Exception catch.");  
        }  
        /* This catch is never reached because  
        ArithmeticException is a subclass of Exception. */  
        catch (ArithmeticException e) { // ERROR - unreachable  
            System.out.println("This is never reached.");  
        }  
    }  
}
```

Nested try Statements

- A **try** statement can be inside the block of another **try**.
- Each time a **try** statement is entered, the context of that exception is pushed on the stack.
- If an inner **try** statement does not have a **catch** handler for a particular exception, the stack is unwound and the next **try** statement's catch handlers are inspected for a match.

Nested try Statements (Cont.)

- This continues until one of the catch statements succeeds, or until all of the nested **try** statements are exhausted.
- If no **catch** statement matches, then the Java run-time system will handle the exception.

```
public class NestedTry {  
    public static void main(String[] args) {  
        int a[] = {1};  
        int x;  
        try {  
            x = 12 / 0;  
            try {  
                a[10] = 5;  
            } catch (ArrayIndexOutOfBoundsException e) {  
                System.out.println("Array Out of Bound");  
            }  
        } catch (ArithmeticException e) {  
            System.out.println("Divide By Zero");  
        }  
    }  
}
```

Nested Try

throw

- Using the throw statement, it is possible for program to throw an exception explicitly.
- The general form of throw is: **throw** *ThrowableInstance*.
- Here, *ThrowableInstance* must be an object of type **Throwable** or a subclass of **Throwable**.
- Primitive types, such as int or char, as well as non-Throwable classes, such as String and Object, cannot be used as exceptions.

throw (Cont.)

- Two ways to obtain a **Throwable** object:
 - Using a parameter in a catch clause.
 - Creating one with the new operator.
- The flow of execution stops immediately after the throw statement.
- The nearest enclosing **try** block is inspected to see if it has a **catch** statement that matches the type of exception.

throw (Cont.)

- If it does find a match, control is transferred to that statement. If not, then the next enclosing **try** statement is inspected, and so on.
- If no matching **catch** is found, then the default exception handler halts the program and prints the stack trace.
- Many of Java's built-in run-time exceptions have at least two constructors: one with no parameter and one that takes a string parameter.

throw (Cont.)

- When the second form is used, the argument specifies a string that describes the exception.

throws

- If a method is capable of causing an exception that it does not handle, it must specify this behavior so that callers of the method can guard themselves against that exception.
- This can be done a **throws** clause in the method's declaration.
- A **throws** clause lists the types of exceptions that a method might throw.

throws(Cont.)

- This is necessary for all exceptions, except those of type **Error** or **RuntimeException**, or any of their subclasses.
- All other exceptions that a method can throw must be declared in the **throws** clause. If they are not, a compile-time error will result.

throw & throws

```
public class ThrowThrowsDemo {  
1   static int divide(int a, int b) throws ArithmeticException {  
2       if (b == 0) {  
3           throw new ArithmeticException();  
4       }  
5       return a / b;  
6   }  
  
7   public static void main(String[] args) {  
8       try {  
9           divide(a: 5, b: 0);  
10      } catch (ArithmeticException e) {  
11          System.out.println(e);  
12      }  
13  }  
}
```

throw & throws

```
class ThrowsDemo {  
    static void throwOne() throws IllegalAccessException {  
        System.out.println("Inside throwOne.");  
        throw new IllegalAccessException("demo");  
    }  
  
    public static void main(String args[]) {  
        try {  
            throwOne();  
        } catch (IllegalAccessException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

finally

- **finally** creates a block of code that will be executed after a **try** /**catch** block has completed and before the code following the **try/catch** block.
- The **finally** block will execute whether or not an exception is thrown.
- If an exception is thrown, the **finally** block will execute even if no catch statement matches the exception.

finally(Cont.)

- Any time a method is about to return to the caller from inside a **try/catch** block, the **finally** clause is also executed just before the method returns.
- This can be useful for closing file handles and freeing up any other resources.
- The **finally** clause is optional.
- Each **try** statement requires at least one **catch** or a **finally** clause.

```
static void A() {  
    try {  
        System.out.println("inside A");  
        throw new RuntimeException("demo");  
    } finally {  
        System.out.println("A's finally");  
    }  
}  
  
static void B() {  
    try {  
        System.out.println("inside B");  
        return;  
    } finally {  
        System.out.println("B's finally");  
    }  
}
```

```
static void C() {  
    try {  
        System.out.println("inside C");  
    } finally {  
        System.out.println("C's finally");  
    }  
}  
  
public static void main(String args[]) {  
    try {  
        A();  
    } catch (Exception e) {  
        System.out.println("Exception caught");  
    }  
    B();  
    C();  
}
```

finally

Java's Unchecked Runtime Exceptions on Subclasses Defined in java.lang

Exception	Meaning
ArithmeticException	Arithmetic error, such as divide-by-zero.
ArrayIndexOutOfBoundsException	Array index is out-of-bounds.
ArrayStoreException	Assignment to an array element of an incompatible type.
ClassCastException	Invalid cast.
EnumConstantNotPresentException	An attempt is made to use an undefined enumeration value.
IllegalArgumentException	Illegal argument used to invoke a method.
IllegalMonitorStateException	Illegal monitor operation, such as waiting on an unlocked thread.
IllegalStateException	Environment or application is in incorrect state.
IllegalThreadStateException	Requested operation not compatible with current thread state.
IndexOutOfBoundsException	Some type of index is out-of-bounds.
NegativeArraySizeException	Array created with a negative size.
NullPointerException	Invalid use of a null reference.
NumberFormatException	Invalid conversion of a string to a numeric format.
SecurityException	Attempt to violate security.
StringIndexOutOfBoundsException	Attempt to index outside the bounds of a string.
TypeNotPresentException	Type not found.
UnsupportedOperationException	An unsupported operation was encountered.

Exception	Meaning
ClassNotFoundException	Class not found.
CloneNotSupportedException	Attempt to clone an object that does not implement the Cloneable interface.
IllegalAccessException	Access to a class is denied.
InstantiationException	Attempt to create an object of an abstract class or interface.
InterruptedException	One thread has been interrupted by another thread.
NoSuchFieldException	A requested field does not exist.
NoSuchMethodException	A requested method does not exist.
ReflectiveOperationException	Superclass of reflection-related exceptions.

Java's Checked Exceptions Defined in **java.lang**

Creating Own Exception Subclasses

- Customize exception types handles situations specific to your applications.
- Just needed to define a subclass of **Exception**.
- The **Exception** class does not define any methods of its own.
- It inherits the methods provided by Throwable.
- The provided methods can be overridden in custom exception classes.

Creating Own Exception Subclasses(Cont.)

- **Exception** defines four public constructors.
 - Two support chained exceptions.
 - The other two are shown here:
 - `Exception( )`
 - `Exception(String msg)`
- The first form creates an exception that has no description.
- The second form lets specify a description of the exception.

Creating Own Exception Subclasses(Cont.)

- Although specifying a description when an exception is created is often useful, sometimes it is better to override **toString()**.
- The version of **toString()** defined by **Throwable** first displays the name of the exception followed by a colon, which is then followed by your description.

Creating Own Exception Subclasses(Cont.)

- By overriding **toString()**, you can prevent the exception name and colon from being displayed. This makes for a cleaner output, which is desirable in some cases.

The Methods Defined by Throwable

Method	Description
final void addSuppressed(Throwable <i>exc</i>)	Adds <i>exc</i> to the list of suppressed exceptions associated with the invoking exception. Primarily for use by the <code>try-with-resources</code> statement.
Throwable fillInStackTrace()	Returns a Throwable object that contains a completed stack trace. This object can be rethrown.
Throwable getCause()	Returns the exception that underlies the current exception. If there is no underlying exception, null is returned.
String getLocalizedMessage()	Returns a localized description of the exception.
String getMessage()	Returns a description of the exception.
StackTraceElement[] getStackTrace()	Returns an array that contains the stack trace, one element at a time, as an array of StackTraceElement . The method at the top of the stack is the last method called before the exception was thrown. This method is found in the first element of the array. The StackTraceElement class gives your program access to information about each element in the trace, such as its method name.
final Throwable[] getSuppressed()	Obtains the suppressed exceptions associated with the invoking exception and returns an array that contains the result. Suppressed exceptions are primarily generated by the <code>try-with-resources</code> statement.
Throwable initCause(Throwable <i>causeExc</i>)	Associates <i>causeExc</i> with the invoking exception as a cause of the invoking exception. Returns a reference to the exception.
void printStackTrace()	Displays the stack trace.
void printStackTrace(PrintStream <i>stream</i>)	Sends the stack trace to the specified stream.
void printStackTrace(PrintWriter <i>stream</i>)	Sends the stack trace to the specified stream.
void setStackTrace(StackTraceElement <i>elements</i> [])	Sets the stack trace to the elements passed in <i>elements</i> . This method is for specialized applications, not normal use.
String toString()	Returns a String object containing a description of the exception. This method is called by <code>println()</code> when outputting a Throwable object.

Custom Exception Subclasses

```
class MyException extends Exception {  
    public String toString() {  
        return "MyException";  
    }  
}  
  
public class CustomExceptionDemo {  
    static void display() throws MyException {  
        throw new MyException();  
    }  
  
    public static void main(String[] args) {  
        try {  
            display();  
        } catch (MyException e) {  
            System.out.println(e);  
        }  
    }  
}
```

Chained Exceptions

- Beginning with JDK 1.4, a feature chained exceptions was incorporated.
- The chained exception feature allows you to associate another exception with an exception. This second exception describes the cause of the first exception.

Chained Exceptions(Cont.)

- For example, a situation in which a method throws an `ArithmeticException` because of an attempt to divide by zero. However, the actual cause of the problem was that an I/O error occurred, which caused the divisor to be set improperly.
- To allow chained exceptions, two constructors and two methods were added to **Throwable**.
 - **Throwable(Throwable *causeExc*)**
 - **Throwable(String *msg*, Throwable *causeExc*)**

Chained Exceptions(Cont.)

- In the first form, *causeExc* is the exception that causes the current exception. That is, *causeExc* is the underlying reason that an exception occurred.
- The second form allows to specify a description at the same time that you specify a cause exception.
- These two constructors have also been added to the `Error`, `Exception`, and `RuntimeException` classes.

Chained Exceptions(Cont.)

- The chained exception methods supported by **Throwable** are
 - **Throwable** `getCause( )`
 - **Throwable** `initCause(Throwable causeExc)`
- The `getCause( )` method returns the exception that underlies the current exception.
- If there is no underlying exception, null is returned.

Chained Exceptions(Cont.)

- The **initCause()** method associates *causeExc* with the invoking exception and returns a reference to the exception.
- The cause exception can be set only once.
- We can call **initCause()** only once for each exception object.

Chained Exceptions(Cont.)

- If the cause exception was set by a constructor, then we can't set it again using **initCause()**.
- **initCause()** is used to set a cause for legacy exception classes that don't support the two additional constructors described earlier.

Chained Exceptions

```
class ChainExcDemo {  
    static void demoproc() {  
        NullPointerException e =  
            new NullPointerException("top layer");  
        e.initCause(new ArithmeticException("cause"));  
        throw e;  
    }  
  
    public static void main(String args[]) {  
        try {  
            demoproc();  
        } catch (NullPointerException e) {  
            System.out.println("Caught: " + e);  
            System.out.println("Original cause: " +  
                e.getCause());  
        }  
    }  
}
```

Three Recently Added Exception Features

- *try-with-resources*
- *multi-catch* allows two or more exceptions to be caught by the same catch clause.
- *final rethrow* or *more precise rethrow* restricts the type of exceptions that can be rethrown.

Using Exceptions

- Exception handling provides a powerful mechanism for controlling complex programs that have many dynamic run-time characteristics.
- **try**, **throw**, and **catch** are clean ways to handle errors and unusual boundary conditions in your program's logic.
- In Java, when a method can fail, we can have it throw an exception. This is a cleaner way to handle failure modes.